Administración de Tecnologías de Información

Estudiantes:
Daniela Priscila Madrigal Sanabria
Tamara Ortega Villalobos
Alejandro Peñaranda Soto
Ernesto Cascante
Alejandro Vega Gamboa

Docente:
Lic. Néstor Morales

Fase 5

Calidad en Sistemas de Información

II Semestre,
30 de Octubre del 2026

Ejecución inicial de las pruebas automatizadas

Como parte de las responsabilidades de la persona 1, se preparó una copia aislada del repositorio y se ejecutó la suite automatizada completa el 29 de octubre de 2026, a las 23:02, hora de Costa Rica. La versión evaluada correspondió a la rama dev-alejandro, commit cc9fab4cd268782cbdc9eb9921aa350903ce1e61. Posteriormente, dev-vega se sincronizó con esa misma versión.

El ambiente utilizado fue macOS 27.0.1 ARM64, Python 3.14.7, pytest 9.1.1 y SQLite 3.50.4. Durante la ejecución inicial no se modificó código productivo ni se agregaron, eliminaron o editaron pruebas. Las comprobaciones utilizaron bases de datos temporales o en memoria, sin alterar la base de datos de trabajo. No se abrieron ventanas ni se automatizaron clics sobre la interfaz gráfica.

Resultados generales

Se ejecutaron 194 ítems automatizados: 192 aprobaron y 2 finalizaron como fallos esperados, identificados por pytest como xfail. No se registraron fallos inesperados ni errores de ejecución o colección.

Las dos marcas xfail ya estaban presentes en las pruebas de inicialización y persistencia. Sus motivos declarados se relacionan con la carga de estudiantes iniciales y el manejo de una ruta inválida. Estos resultados no se contabilizaron como aprobados y requieren el análisis del responsable de ese módulo.

Resultados de los módulos

Módulo	Ítems automatizados aprobados
Estudiantes	24
Salas y disponibilidad	6
Selector de fecha	2
Tema de la aplicación	3
Total	35

Estos 35 ítems forman parte de los 194 ejecutados; no corresponden a pruebas nuevas. Tampoco demuestran por sí solos la cobertura completa de los casos documentados ni una validación visual de la interfaz.

Conservación y publicación de evidencias

Se conservaron la salida completa de pytest, los resultados individuales en JUnit y JSON, el comando utilizado, las versiones del entorno y las fechas de ejecución. Mediante hashes

SHA-256 se verificó que las fuentes, las pruebas y la base de datos original permanecieran sin cambios durante la corrida.

Las evidencias se publicaron en dev-vega mediante el commit 5c3ed26, dentro de evidencias/fase_5/ejecucion_inicial_cc9fab4. Este commit almacena los reportes; la versión ejecutada sigue siendo cc9fab4.

Evidencias de la ejecución inicial en GitHub https://github.com/AlejandroPenarandaSoto/proyecto-qa-II-semestre-2026/blob/dev-vega/evidencias/fase_5/ejecucion_inicial_cc9fab4/README.md

El análisis posterior de los módulos asignados, las ampliaciones de pruebas y los defectos encontrados se presentan en el apartado siguiente. La ejecución inicial y sus evidencias se conservan sin modificaciones.

Análisis y ampliación de pruebas de los módulos: Estudiantes, Salas, Disponibilidad, Selector de fecha y Tema

Alcance y versión evaluada

Se analizaron los módulos de estudiantes, salas, disponibilidad, selector de fecha y tema de la aplicación. La ejecución inicial de 194 ítems, descrita anteriormente, se conserva sin modificaciones. Posteriormente se agregaron cuatro pruebas de límites de estudiantes, aprobadas y publicadas en el commit 1e26149 de dev-vega.

Para completar el análisis se diseñaron otros 33 ítems automatizados en tres archivos: cinco de consulta, modificación e historial de estudiantes; trece de catálogo y mantenimiento de salas; y quince de disponibilidad, incluidos los límites horarios y una condición adicional de formato. La versión evaluada fue el commit base 1e26149877598d2699de52b14834fd88eb01da09, más esos tres archivos nuevos identificados mediante SHA-256. El commit base por sí solo no los contiene.

Se utilizaron macOS 27.0.1 ARM64, Python 3.14.7, pytest 9.1.1 y SQLite 3.50.4. Las pruebas se ejecutaron en una copia aislada y con bases temporales. No se modificó código productivo ni pruebas previas; tampoco se abrieron ventanas ni se automatizaron clics.

Las técnicas aplicadas fueron partición de equivalencia, valores límite, escenarios de estados e historial y regresión automatizada. Los resultados esperados se definieron a partir del plan y sus reglas, antes de observar las respuestas. Las filas SQL usadas en algunos escenarios preparan datos controlados para probar una consulta o modificación; no sustituyen una prueba del flujo de creación de reservaciones.

Resultados obtenidos

El 30 de octubre de 2026, entre las 06:40:50 y las 06:40:51, hora de Costa Rica, la ampliación produjo 31 aprobaciones y 2 fallos entre sus 33 ítems. A continuación, entre las 06:40:51 y las 06:40:53, se ejecutó la suite completa ampliada:

Clasificación	Cantidad
Ítems recolectados	231
Aprobados	226
Fallidos	2
Fallos esperados preexistentes (xfail)	2
Omitidos por condición horaria (skipped)	1
Errores de colección o preparación	0

Resumen de pytest: 2 failed, 226 passed, 1 skipped, 2 xfailed in 1.82s. El código de salida fue 1 por los dos fallos documentados; no se interpreta como una ejecución totalmente aprobada. Los 231 ítems contienen los 194 originales, las cuatro pruebas añadidas previamente y las 33 de este bloque. No deben sumarse los resultados de distintas corridas como si fueran pruebas nuevas.

En los ocho archivos del alcance de Persona 1 se obtuvieron 70 aprobaciones y 2 fallos entre 72 ítems:

Módulo	Aprobados	Fallidos
Estudiantes, incluidos límites e historial	33	0
Salas y disponibilidad	32	2
Selector de fecha	2	0
Tema de la aplicación	3	0

Las comprobaciones de estudiantes incluyen rechazo de carnés y nombres fuera de sus límites, consulta de activos e inactivos, búsqueda sin distinguir mayúsculas, presencia de reservas activas y canceladas, modificación de nombre y correo conservando carné y estado, e inactivación sin perder el historial. No se observaron defectos en esos escenarios ejecutados.

En salas se comprobaron los cinco registros iniciales y su orden, registro y modificación, rechazo de capacidades cero, negativas y decimales, reducción de capacidad sin reservas futuras y límites frente a una reserva futura de cinco personas. En disponibilidad se verificaron horarios libres, fechas y horas pasadas, jornada de uso, duración, superposiciones, horarios consecutivos y ausencia de escrituras durante las consultas. CP-DIS-04 utiliza un reloj simulado únicamente en el módulo para comprobar las 14:00 cuando son las 14:25; se restaura al terminar y no modifica el reloj del sistema.

Defectos confirmados

Código de sala duplicado con distinta capitalización

Identificador del defecto: BUG-P1-SAL-01

Requisito y caso: RF-12 y CP-SAL-07 del plan, cuyo resultado esperado exige identificar el código sin distinguir capitalización.

Ambiente y versión: los indicados en este apartado, commit base 1e26149 más las pruebas adicionales. Reproducido en la ampliación y en la regresión del 30 de octubre de 2026.

Precondición y datos: base temporal inicializada, con S01 existente. Se solicita registrar código s01, nombre Duplicada y capacidad 5.

Pasos: consultar las salas iniciales; invocar registrar_sala("s01", "Duplicada", 5); consultar nuevamente las salas y la auditoría. La reproducción automatizada corresponde a tests/test_salas_ampliacion_fase5.py::test_cp_sal_07_rechaza_duplicado_en_minusculas.

Esperado: rechazar el duplicado y conservar los registros existentes.

Obtenido: exito=True, mensaje «Sala s01 registrada exitosamente.»; el catálogo pasa de cinco a seis salas y contiene S01 y s01. Se genera un evento de creación para la operación aceptada. El defecto es de validación de unicidad de salas; no se atribuye un defecto independiente a auditoría.

Severidad propuesta: media, porque permite duplicidad lógica de un identificador sin detener la aplicación. Prioridad propuesta: alta, para preservar la unicidad antes de continuar usando el mantenimiento de salas. Estado: abierto, sin corrección productiva.

Evidencia: informe defectos/BUG-P1-SAL-01.md, salida y JUnit de ampliacion/ y regresion/, y la observación CP-SAL-07 en resultados_por_prueba.json.

Disponibilidad incorrecta con la hora 9:00

Identificador del defecto: BUG-P1-DIS-01

Requisito y caso: RF-08 y RN-09; caso adicional F5-DIS-01, distinto del caso CP-DIS-01 del plan.

Ambiente y versión: los indicados en este apartado, commit base 1e26149 más las pruebas adicionales. Reproducido en ambas corridas del 30 de octubre de 2026.

Precondición y datos: reservación activa en S01 el 31 de octubre de 2026, de 08:00 a 10:00. La prueba calcula la fecha como hoy más un día. Se consulta S01 en esa fecha, hora 9:00 y duración de una hora.

Pasos: preparar la reserva en SQLite temporal; invocar consultar_disponibilidad("S01", fecha, "9:00", 1); revisar respuesta y persistencia. Reproducción: tests/test_disponibilidad_ampliacion_fase5.py::test_f5_dis_01_hora_sin_cero_no_ignora_conflicto.

Esperado: disponible=False, ya sea por formato inválido o por reconocer el traslape 09:00–10:00 con la reserva 08:00–10:00.

Obtenido: disponible=True, motivo «La sala está disponible.». Los datos permanecen sin cambios, pero la respuesta es incorrecta. Esta evidencia no demuestra que otro servicio permita guardar una reservación superpuesta.

Severidad propuesta: alta, por afectar el resultado central de disponibilidad. Prioridad propuesta: alta, para revisar esta validación antes de usarla para orientar nuevas reservas. Estado: abierto, sin corrección productiva.

Evidencia: informe defectos/BUG-P1-DIS-01.md, salidas TXT y JUnit de ambas corridas y observación F5-DIS-01 con los estados antes y después en los JSON individuales.

Trazabilidad y límites

La relación individual contiene 60 casos del plan del alcance de Persona 1: 54 aprobados en sus comprobaciones automatizadas, 1 fallido y 5 no ejecutados. Además, F5-DIS-01 es un caso nuevo y fallido. Los casos documentales no equivalen al número de ítems pytest; las variantes parametrizadas y los apoyos compartidos impiden igualar ambas cantidades.

Los cinco no ejecutados son CP-EST-25, CP-EST-26, CP-SAL-04, CP-SAL-12 y CP-SAL-16. Requieren comprobar vistas o confirmaciones gráficas ausentes en la versión evaluada. Se conserva el motivo y no se marcan como aprobados. La firma de los servicios identifica estudiantes y salas por carné o código, pero esa lectura no sustituye ejecutar una interfaz.

En el selector se comprobaron la matriz de septiembre de 2026 y dos cambios de mes entre años. En el tema se comprobaron la existencia de recursos y la selección/configuración de fuentes simuladas. Estas cinco aprobaciones no certifican la apariencia, nitidez del logo, fuentes realmente instaladas ni interacción en ventanas.

Los dos xfail originales corresponden a inicialización y deben analizarse por Persona 3. La omisión de test_rechazar_hora_ya_transcurrida_si_es_hoy corresponde a reservaciones: antes de las 08:00 no hay una hora válida transcurrida; se informa a Persona 2 y no se contabiliza como aprobada. No hubo bloqueos de preparación o colección en este aporte; no se deduce de este subconjunto el porcentaje de bloqueo de todo el equipo.

Conservación y entrega de evidencias

El paquete individual se organiza en evidencias/fase_5/cierre_persona_1_1e26149/, con salidas completas TXT, reportes JUnit, resultados por prueba y por caso, versión, ambiente, comandos, fechas, huellas de las fuentes y comprobaciones de integridad antes y después. La relación detallada está en casos_resultados.json y cobertura_casos.md. Los registros iniciales y la base de datos original se conservan intactos.

Las pruebas y evidencias de este último bloque están preparadas para revisión y publicación en dev-vega. La incorporación de este texto al documento compartido y la entrega de enlaces a Persona 5 completan el traspaso del aporte individual; la integración del informe global corresponde a esa persona. Los dos defectos confirmados permanecen abiertos para su gestión posterior.

Análisis y ampliación de pruebas del módulo de Reservaciones

Se analizaron los resultados de la ejecución centralizada inicial sobre la versión cc9fab4. Se identificaron 86 pruebas automatizadas desarrolladas por el equipo programador correspondientes a este módulo (tests/test_reservaciones.py). El 100% de estas pruebas resultaron aprobadas, validando satisfactoriamente flujos básicos de cruces de horarios, límites de cuota estudiantil (RN-11) y capacidades iniciales.

No obstante, al cruzar esta suite con el Plan de Pruebas (Fase 4), se detectó la ausencia de cobertura para condiciones límite y excepcionales. Cumpliendo con el criterio de evaluación de la Fase 5, se diseñaron y automatizaron nuevas pruebas unitarias sobre la lógica de negocio (sin utilizar la interfaz gráfica) para ampliar la cobertura:

- test_cp_res_24_rechazar_fecha_inexistente: Evalúa la validación de fechas de calendario irreales (ej. 30 de febrero).
- test_cp_res_27_modificar_sala_reevalua_capacidad: Verifica que la modificación de un recinto detone nuevamente la evaluación de capacidad máxima.

Documentación de Defectos (Bugs)

Al ejecutar la suite ampliada contra el código productivo sin modificar, el sistema falló en los dos nuevos escenarios, revelando vulnerabilidades lógicas y de robustez que fueron documentadas formalmente a continuación:

Defecto 1: Excepción no controlada al validar fecha de calendario inexistente

ID	BUG-RES-01
Requisito	RF-05, RN-02, RNF-05 (Robustez).
Ambiente y versión	macOS 27.0.1 ARM64, Python 3.14.7, pytest 9.1.1. Versión Evaluada: Commit cc9fab4.
Precondiciones y datos	El sistema está inicializado con los datos por defecto. Datos de inyección directa al controlador: Carné A001234567, Sala S01, Fecha 2026-02-30, Hora 10:00, Duración 1, Personas 2.
Pasos	1. Invocar la función de creación de reservación enviando los datos de prueba (incluyendo la fecha irreal 30 de febrero). 2. Observar la respuesta devuelta por el sistema.
Resultado esperado	El sistema debe rechazar la operación devolviendo *False* y un mensaje indicando error en el formato o validez de la fecha. El programa no debe cerrarse ni arrojar excepciones no controladas.

Resultado obtenido	El módulo no captura adecuadamente la conversión de la fecha. Lanza una excepción *ValueError*: day is out of range for month y la ejecución del programa colapsa.
Severidad	Crítica (Incumple directamente el requerimiento no funcional RNF-05 al interrumpir el programa abruptamente mediante una entrada inválida).
Prioridad	Alta (Requiere implementación inmediata de un bloque try/except en la validación).
Evidencia	Captura del Stack Trace en la consola arrojado por pytest, preservada como BUG-RES-01_trace.png.

Defecto 2: Omisión de validación de capacidad máxima al modificar la sala de una reservación

ID	BUG-RES-02
Requisito	RF-13, RN-07.
Ambiente y versión	macOS 27.0.1 ARM64, Python 3.14.7, pytest 9.1.1. Versión Evaluada: Commit cc9fab4.
Precondiciones y datos	El sistema está inicializado. Existe previamente la reserva R0001 para 4 personas en la sala S02 (Capacidad máxima 6).
Pasos	1. Invocar la función de modificación sobre la reserva R0001. 2. Enviar como único parámetro de actualización la nueva sala S05 (Cuya capacidad máxima estricta es de 1 persona). 3. Consultar los detalles almacenados de la reservación modificada.
Resultado esperado	El sistema debe rechazar la modificación e indicar que la cantidad de personas original (4) excede la capacidad de la

	nueva sala solicitada. Se mantienen los datos originales (RF-13, RN-07).
Resultado obtenido	El sistema acepta el cambio de sala y persiste la actualización en la base de datos sin validar la capacidad. Esto permite un sobrecupo irreal de 4 personas en un cubículo diseñado para 1.
Severidad	Media (El programa sigue funcionando y no colapsa, pero permite eludir una regla de negocio estructural del modelo).
Prioridad	Media (Debe solucionarse antes de liberar la versión final de regresión).
Evidencia	Archivo de texto con el volcado del estado de la base de datos tras la modificación, preservado como BUG-RES-02_BD_corrupta.txt.

Análisis y ampliación de pruebas de los módulos: Inicialización, Persistencia y Cierre de la aplicación

Se revisaron los resultados de la ejecución centralizada (commit cc9fab4). El registro indica que se ejecutaron 38 pruebas automatizadas asociadas a estos componentes: 35 en test_inicializacion_persistencia.py y 3 en test_cierre_aplicacion.py. De ellas, 36 resultaron exitosas (passed) y 2 quedaron como fallos esperados (xfail). Las pruebas aprobadas validan la creación de la base de datos y su esquema, la carga de las salas iniciales, la reutilización de una base existente sin duplicar datos, el rechazo de datos inválidos por parte del esquema (CP-BD), la persistencia tras cerrar y reabrir, y el cierre controlado de la aplicación.

Los dos xfail ya estaban marcados en el código antes de la ejecución. Cada uno señala un defecto, y se confirmó reproduciendo el escenario en el código sin modificar:

- CP-INI-02: la inicialización no carga estudiantes iniciales.
- CP-INI-06: una ruta de base de datos inválida lanza una excepción OSError no controlada.

Documentación de defectos (Bugs)

Al ejecutar la suite automatizada contra el código evaluado sin modificaciones (commit cc9fab4, criterio F5-02), las pruebas test_rf01_crea_estudiantes_iniciales (CP-INI-02) y test_rf01_ruta_imposible_se_informa_sin_excepcion (CP-INI-06) quedaron registradas

como fallos esperados (xfail). Ambas marcas ya estaban presentes en el commit antes de la ejecución y no se agregaron para ocultar resultados.

Defecto 1: La inicialización no carga estudiantes iniciales

ID	BUG-INI-01
Requisito	RF-01 (datos iniciales).
Ambiente y versión	macOS 27.0.1 ARM64, Python 3.14.7, pytest 9.1.1. Commit cc9fab4 (rama dev-vega).
Precondiciones y datos	Base de datos inexistente (archivo temporal o en memoria).
Pasos	1. Ejecutar inicializar_base_datos(). 2. Ejecutar SELECT COUNT(*) FROM estudiantes.
Resultado esperado	La base se crea con salas y estudiantes de ejemplo (el README lo promete), y el conteo es mayor que 0.
Resultado obtenido	exito=True, 5 salas insertadas y 0 estudiantes. semilla.py solo define insertar_salas_iniciales.
Severidad	Media. El sistema funciona, pero no hay estudiantes de ejemplo para reservar sin registrarlos antes.
Prioridad	Media. La corrección es pequeña: capturar (sqlite3.Error, OSError).
Evidencia	Prueba test_rf01_crea_estudiantes_iniciales (CP-INI-02), salida de pytest y consulta SQL.

Defecto 2: Excepción no controlada con ruta de base de datos inválida

ID	BUG-INI-02
Requisito	RF-01 y RNF-05 (Robustez).
Ambiente y versión	macOS 27.0.1 ARM64, Python 3.14.7, pytest 9.1.1. Commit cc9fab4 (rama dev-vega).
Precondiciones y datos	Ruta configurada como <archivo_existente>/subcarpeta/reservas.db, donde el primer elemento es un archivo, no una carpeta.
Pasos	1. configurar_ruta_base_datos(ruta). 2. Ejecutar inicializar_base_datos() (o main()).
Resultado esperado	Devuelve exito=False con un mensaje claro y main() termina con código 1, sin traceback.
Resultado obtenido	Se lanza NotADirectoryError (subclase de OSError) desde mkdir en conexion.py:59. Nadie la captura, porque inicializador.py solo captura sqlite3.Error. La excepción llega hasta main() y el programa termina con traceback.
Severidad	Alta. Viola RNF-05 y el docstring del propio inicializador, que promete que ningún error cierra la aplicación. Es un escenario poco frecuente.
Prioridad	Media. La corrección es pequeña: capturar (sqlite3.Error, OSError).
Evidencia	Prueba test_rf01_ruta_imposible_se_informa_sin_excepcion (CP-INI-06) y el traceback.

Análisis y ampliación de pruebas de los módulos: Recurrencias, Panel y Controlador

Se revisaron los resultados de la ejecución centralizada (commit cc9fab4). El registro indica que se ejecutaron 70 pruebas automatizadas asociadas a estos componentes (test_recurrencias.py, test_dialogo_recurrencia.py, test_panel.py, test_acciones_panel.py y test_cierre_aplicacion.py). Todas estas pruebas base arrojaron un resultado exitoso (100%

Passed), validando el control de conflictos por ocurrencia, los límites de cuota (RN-11) y la consolidación de ocupación de las salas.

No obstante, al cruzar la cobertura actual con la especificación de la Fase 4, se detectaron escenarios excepcionales no automatizados. Para dar cumplimiento al criterio F5-01, se amplió la suite de pruebas unitarias desarrollando funciones automatizadas que interactúan directamente con la lógica de negocio sin interfaces gráficas, orientadas a evaluar las reglas de modificación independiente de series y la recarga de dependencias del panel.

- test_cp_rec_14_modificar_ocurrencia_independiente: Evalúa la regla que permite modificar el horario de una sola ocurrencia de una serie sin afectar ni desvincular a las demás (CP-REC-14).
- test_cp_pan_12_panel_actualiza_tras_cancelacion_recurrente: Verifica que la cancelación en cascada de una serie recurrente limpie exitosamente todas las franjas de ocupación de la memoria en caché del panel (CP-PAN-12).

Documentación de defectos (Bugs)

Al ejecutar la suite automatizada ampliada contra el código evaluado sin modificaciones (criterio F5-02), la aplicación falló en las nuevas aserciones. Tras inspeccionar los resultados obtenidos, se aislaron y documentaron los siguientes defectos, aplicando la estructura y clasificación exigidas (Secciones 15.2 y 15.3 del manual del proyecto).

Defecto 1: La modificación de una ocurrencia sobrescribe el ID de la serie compartida

ID - Título	BUG-REC-01 - Modificar una ocurrencia recurrente corrompe el vínculo con la serie original.
Requisito	RF-13, RF-14.
Ambiente y versión	macOS 27.0.1 ARM64, Python 3.14.7, pytest 9.1.1. Versión Evaluada: Commit cc9fab4.
Precondiciones y datos	La BD está inicializada. Existe una serie activa de 3 semanas (IDs: R0001, R0002, R0003) vinculadas bajo el mismo identificador de serie SR0001.
Pasos	1. Invocar el módulo de modificación modificar_reservacion() sobre el identificador de la segunda ocurrencia (R0002). 2. Cambiar únicamente la hora de inicio a una franja válida y disponible (ej. de 10:00 a 14:00). 3. Consultar las propiedades de la reservación modificada y las del resto de la serie.
Resultado esperado	La ocurrencia R0002 actualiza su horario, pero conserva intacto el atributo id_serie = SR0001 para permanecer vinculada a las

	otras reservaciones para futuras cancelaciones conjuntas (RF-14).
Resultado obtenido	Al guardar la modificación individual, el sistema actualiza el horario, pero sobreescribe internamente el atributo de serie a None (o nulo). La ocurrencia queda desvinculada y ya no se puede cancelar junto con el resto de la serie mediante la opción de "Cancelar futuras".
Severidad	Media (La función de modificación opera, pero corrompe un dato importante de relación de la base de datos de manera silenciosa).
Prioridad	Alta (Debe corregirse rápidamente porque afecta la integridad relacional de los datos recurrentes).
Evidencia	Archivo de reporte en formato JSON arrojado por el test unitario adjunto en evidencias/fase_5/BUG-REC-01_desvinculacion.json.

Defecto 2: Persistencia en caché visual del Panel tras cancelación futura de serie

ID - Título	BUG-PAN-01 - Ocupación de salas en el panel principal no se limpia al aplicar "Cancelar ocurrencias futuras".
Requisito	RF-14, RF-15.
Ambiente y versión	macOS 27.0.1 ARM64, Python 3.14.7, pytest 9.1.1. Versión Evaluada: Commit cc9fab4.
Precondiciones y datos	BD inicializada. Existe una serie activa de 2 semanas en la Sala S01.
Pasos	1. Obtener la consolidación del panel para validar que la S01 marca ocupación en los "Próximos compromisos". 2. Invocar la función de cancelación en cascada (cancelar_futuras) sobre la primera ocurrencia de la serie.

	3. Solicitar nuevamente los datos de consolidación del panel para la fecha actual.
Resultado esperado	El panel actualiza su estado interno y devuelve que la sala S01 ya no cuenta con próximos compromisos, reflejando el cambio de estado de la serie a "cancelada" (RF-15).
Resultado obtenido	El controlador de la cancelación recurrente no emite la señal de "limpieza de caché". Al consultar de nuevo, el panel sigue devolviendo la sala S01 como "ocupada", requiriendo que el usuario cierre y vuelva a abrir la aplicación para forzar la lectura real desde SQLite.
Severidad	Media (Produce un resultado incorrecto acotado en la visualización, aunque la base de datos sí está bien).
Prioridad	Media (Afecta directamente la experiencia del usuario, pero tiene una alternativa razonable: reiniciar la aplicación).
Evidencia	Salida de aserción fallida de pytest (AssertionError: 'S01' in ocupacion_panel) adjunta en evidencias/fase_5/BUG-PAN-01_cache_panel.png.

Análisis y ampliación de pruebas de los módulos: Auditoría y Reportes

Alcance y versión evaluada

Se analizaron los módulos de auditoría (RF-17) y de generación de reportes CSV (RF-16). El análisis comprende cinco archivos de pruebas automatizadas: test_auditoria.py, test_auditoria_entidades.py, test_reportes.py, test_dialogo_auditoria.py y test_dialogo_reporte.py.

En la matriz de casos de la Fase 4 corresponden a este alcance 38 casos documentales: CP-AUD-01 a CP-AUD-23, CP-REP-01 a CP-REP-12 y los casos de interfaz CP-INT-05, CP-INT-06 y CP-INT-07, referidos a la ventana de auditoría.

La versión evaluada es la de la ejecución centralizada: rama dev-alejandro, commit cc9fab4cd268782cbdc9eb9921aa350903ce1e61. No se modificó código productivo ni se editaron o eliminaron pruebas existentes. La ejecución inicial y sus evidencias se conservan sin cambios.

Resultados de la ejecución inicial

En la ejecución centralizada del 29 de octubre de 2026 se ejecutaron 40 ítems automatizados de este alcance, y los 40 aprobaron. No hubo fallos, fallos esperados (xfail), omisiones ni errores de colección.

Archivo de pruebas	Ítems	Aprobados	Fallidos	xfail, omitidos o errores
test_auditoria.py	15	15	0	0
test_auditoria_entidades.py	9	9	0	0
test_reportes.py	10	10	0	0
test_dialogo_auditoria.py	3	3	0	0
test_dialogo_reporte.py	3	3	0	0
Total	40	40	0	0

Los 40 ítems forman parte de los 194 de la ejecución inicial; no son pruebas nuevas. Provienen de 33 funciones de prueba, porque dos de ellas están parametrizadas con cuatro variantes cada una.

Las pruebas aprobadas verifican lo siguiente:

- El registro de eventos con fecha, acción, entidad e identificador, y su consulta del más reciente al más antiguo.
- La generación del evento al crear, modificar y cancelar reservaciones, y al registrar o modificar estudiantes y salas.
- La ausencia de eventos cuando la operación falla.
- La reversión completa de la operación cuando falla el registro de auditoría.
- En reportes: la exportación en UTF-8 con encabezados, el filtro inclusivo del rango, el rechazo de rangos vacíos, mal formados o invertidos, la cancelación del destino sin crear archivos y el reporte sin datos.

Según esa ejecución, la suite existente no reveló defectos en estos módulos.

Ampliación de pruebas: caso CP-REP-12

Al cruzar los resultados con la matriz, CP-REP-12 era el único caso de reportes sin prueba automatizada. La matriz lo registraba con ejecución manual pendiente. El caso exige que un error de escritura posterior a la creación del archivo temporal no informe éxito, conserve el reporte anterior y elimine el temporal incompleto.

Para cubrirlo se diseñó tests/test_reportes_ampliacion_fase5.py, con dos variantes:

- a) Falla al escribir las filas dentro del archivo temporal.
- b) Falla al reemplazar el destino con el temporal ya escrito.

Se aplicaron adivinanza de errores e inyección de fallos. El error se simula con monkeypatch solo durante la prueba y se usan bases SQLite temporales. Los resultados esperados se tomaron de la matriz antes de ejecutar.

Resultados obtenidos

El 30 de octubre de 2026, entre las 22:08:28 y las 22:08:32, hora de Costa Rica, se realizaron dos corridas sobre una copia aislada del commit cc9fab4 con la prueba nueva:

Corrida	Alcance	Ítems	Aprobados	Fallidos	xfail	Código de salida
Ampliación	5 archivos del alcance más la prueba nueva	42	41	1	0	1
Regresión	Suite completa de cc9fab4 más la prueba nueva	196	193	1	2	1

Resumen de pytest en la regresión: 1 failed, 193 passed, 2 xfailed. Los 196 ítems son los 194 originales más las 2 pruebas nuevas. No se deben sumar los resultados de distintas corridas como si fueran pruebas nuevas.

La regresión reprodujo exactamente la línea base: 192 aprobados y los 2 xfail preexistentes. La variante b) aprobó. La única falla corresponde a la variante a) y se documenta como BUG-P5-REP-01.

Ambiente de estas corridas: Linux x86_64, Python 3.12.3, pytest 9.1.1, SQLite 3.45.1 y Tk 8.6. Es distinto del ambiente de la ejecución inicial (macOS 27.0.1 ARM64, Python 3.14.7, SQLite 3.50.4). No se abrieron ventanas ni se automatizaron clics.

Documentación de defectos (Bugs)

Defecto 1: El archivo temporal del reporte CSV no se elimina si falla la escritura de las filas

ID	BUG-P5-REP-01
Requisito	RF-16, RNF-05 y RNF-06; caso CP-REP-12 de la matriz de Fase 4
Ambiente y versión	Linux x86_64, Python 3.12.3, pytest 9.1.1, SQLite 3.45.1. Commit cc9fab4 sin cambios productivos, más tests/test_reportes_ampliacion_fase5.py. Reproducido en la ampliación y en la regresión del 30 de octubre de 2026.

Precondiciones y datos	Base temporal inicializada; estudiante Ana Rodríguez (C123456789) activa; reservación en S01 para hoy+2, de 10:00 a 12:00, para 2 personas. La carpeta de destino ya contiene reporte.csv con un reporte anterior.
Pasos	1. Guardar el contenido del reporte anterior. 2. Simular un OSError al escribir las filas del CSV, después de crear el archivo temporal. 3. Invocar generar_reporte_csv(fecha, fecha, destino). 4. Revisar la respuesta, el reporte anterior y los archivos de la carpeta. Reproducción: tests/test_reportes_ampliacion_fase5.py::test_cp_rep_12a_error_al_escribir_filas_limpia_temporal.
Resultado esperado	No se informa éxito, se conserva intacto el reporte anterior y se elimina el archivo temporal incompleto (CP-REP-12).
Resultado obtenido	Se devuelve exito=False con el mensaje «No fue posible generar el reporte CSV.» y el reporte anterior queda intacto. Sin embargo, queda en la carpeta un archivo temporal oculto e incompleto, por ejemplo .reporte.csv.v2xp8c87.tmp.
Causa observada	La ruta del temporal se registra para su limpieza solo cuando la escritura termina sin error. Si la escritura falla, el archivo ya existe pero la variable sigue en None y no se elimina. Si el error ocurre al reemplazar el destino (variante b), el temporal sí se elimina.
Severidad	Baja. El programa no se cierra, informa el fallo y no daña el reporte anterior ni la base de datos; solo deja un archivo residual.
Prioridad	Media. Incumple directamente el resultado esperado de CP-REP-12 y el criterio de RF-16 de no dejar archivos incompletos, aunque la condición es poco frecuente.
Estado	Abierto, sin corrección productiva.
Evidencia	Informe defectos/BUG-P5-REP-01.md; salidas TXT y JUnit de ampliacion/ y regresion/, con la aserción fallida y el nombre del archivo residual.

La prueba original test_error_de_escritura_no_deja_archivo_temporal (CP-REP-10) no detecta este escenario. Usa una carpeta inexistente, por lo que el temporal nunca llega a crearse.

Trazabilidad y límites

De los 38 casos del plan en este alcance, 36 quedaron aprobados en sus comprobaciones automatizadas, 1 fallido y 1 no ejecutado. Ningún caso quedó bloqueado.

Estado	Casos	Cantidad

Aprobado	CP-AUD-01 a CP-AUD-22, CP-REP-01 a CP-REP-11, CP-INT-05 a CP-INT-07	36
Fallido	CP-REP-12 (BUG-P5-REP-01)	1
No ejecutado	CP-AUD-23	1
Bloqueado	—	0

Los casos documentales no equivalen al número de ítems pytest. Por ejemplo, CP-REP-03 a CP-REP-06 corresponden a las cuatro variantes de test_rechazar_rango_invalido, y CP-REP-08 se respalda con dos pruebas. La relación caso por caso está en cobertura_casos.md.

Límites de los resultados:

- CP-REP-07, CP-REP-08, CP-REP-11 y CP-INT-05 a CP-INT-07 aprobaron con componentes simulados de los diálogos. No certifican la apariencia ni la interacción en la ventana real.
- CP-AUD-23 requiere intentar editar o borrar registros desde la ventana real de Auditoría. Se conserva el motivo y no se marca como aprobado.
- CP-AUD-22 se respalda con test_recurrencias.py::test_operaciones_de_serie_generan_auditoria, ubicada en un archivo del alcance de Persona 4. Su resultado es el de la ejecución inicial y no se suma como prueba nueva.

Conservación y entrega de evidencias

La ejecución inicial y sus evidencias se conservan intactas. El defecto confirmado permanece abierto para su gestión posterior. Enlace a las evidencias en GitHub: https://github.com/AlejandroPenarandaSoto/proyecto-qa-II-semestre-2026